ASSIGNMENT 2 – CO2

Q4: CHOOSING A MAP FOR A PHARMACY INVENTORY

Hospital Pharmacy Stock Lookup

Objective

Compare HashMap, TreeMap and LinkedHashMap for medicine stock lookup and two reports: ascending medicine-code order and insertion order. Validate the design using three inventory sizes and repeated measurements.

This report intentionally excludes student-identifying details.

1. Problem and Experimental Design

A hospital pharmacy identifies each medicine by a unique medicine code. Pharmacists perform frequent lookups, while the evening process requires one report sorted by code and another in insertion order. The benchmark tests 1,000, 10,000 and 50,000 medicines. Each measured lookup repetition performs 100,000 lookups. Five measured repetitions are collected for lookup and both report-generation operations. Map population time is also measured.

The dataset is inserted in a permuted code order so insertion order differs from ascending code order. The same medicine records and lookup-index sequence are used for all map implementations at each size. The program verifies ordering before measurement.

2. Predictions Before Implementation

Map	put()	get()	Iteration / natural order
HashMap	O(1) average; O(n) worst	O(1) average; O(n) worst	O(n); no order guarantee
TreeMap	O(log n)	O(log n)	O(n); sorted keys
LinkedHashMap	O(1) average; O(n) worst	O(1) average; O(n) worst	O(n); insertion order

HashMap is expected to provide fast average lookup but cannot guarantee either report order. TreeMap naturally produces code order at logarithmic lookup/update cost. LinkedHashMap naturally produces insertion order and offers average constant-time lookup, but needs sorting for code order.

3. Report Generation Strategy

Code-order report: TreeMap iterates directly in ascending key order. HashMap and LinkedHashMap copy entries and sort by medicine code, requiring O(n log n) time and O(n) temporary space.

Insertion-order report: LinkedHashMap iterates directly in insertion order. HashMap and TreeMap use an auxiliary list of medicine codes retained at insertion time; iterating it and looking up values takes O(n) expected time and O(n) additional key storage.

4. Experimental Results

The table values below are average milliseconds from the prepared Java 21 run, calculated over five repetitions. They are illustrative measurements from this execution environment; your own computer may produce different timings.

Inventory size: 1,000 medicines

Map	100k lookups (ms)	Code report (ms)	Insertion report (ms)
HashMap	2.300	0.809	0.105
TreeMap	13.628	0.263	0.205
LinkedHashMap	2.485	0.907	0.154

Inventory size: 10,000 medicines

Map	100k lookups (ms)	Code report (ms)	Insertion report (ms)
HashMap	3.166	4.703	1.040
TreeMap	25.969	0.618	1.942
LinkedHashMap	3.027	5.088	0.270

Inventory size: 50,000 medicines

Map	100k lookups (ms)	Code report (ms)	Insertion report (ms)
HashMap	7.637	16.029	2.631
TreeMap	42.560	1.853	16.739
LinkedHashMap	3.852	21.250	0.272

The raw per-repetition measurements are in task4_results.csv. The console summary also includes average population time and nanoseconds per operation. Repeated averaged measurements, not one isolated timing, are the primary evidence.

5. Scalability and Deployment Decision

Acceptability criterion: for this coursework prototype, define an acceptable average stock lookup as no more than 1 millisecond per lookup under the tested workload. Divide the measured 100,000-lookup batch time by 100,000 to obtain the average per lookup. This is an explicit experimental target, not a universal hospital-system service-level requirement.

The remaining lookup cost comes from key hashing/comparison, map access, JVM execution, memory access and machine load. TreeMap additionally follows a logarithmic search path. HashMap and LinkedHashMap are expected to provide average O(1) lookup when hashes are well distributed.

Recommended deployment: LinkedHashMap is a practical primary map when frequent lookups and the insertion-order report are both required; sort a snapshot of entries when producing the evening code-order report. If code-order reporting is dominant and sorted order must always be maintained, choose TreeMap. If only lookup speed matters and report order can be built separately, HashMap is also suitable. The final choice should follow the workload frequency and measurements.

Why Hashtable is not preferred

Hashtable is a legacy synchronized map. Synchronization can add contention overhead, and modern Map implementations provide more flexible choices. For new code, select HashMap, TreeMap or LinkedHashMap according to ordering and performance requirements. For concurrent access, evaluate ConcurrentHashMap and the actual consistency requirements rather than choosing Hashtable by default.

6. Alternative: Sorted Array + Arrays.binarySearch()

Keep a Medicine[] sorted by medicine code. Arrays.binarySearch() finds a code in O(log n) time. Insertion finds the position and shifts later elements, taking O(n). The code-order report is already sorted and takes O(n) to traverse. Array storage is O(n); lookup needs O(1) auxiliary space.

Approach	Lookup	Insert / put	Code report	Space
HashMap	O(1) avg; O(n) worst	O(1) avg; O(n) worst	O(n log n) sort	O(n)
TreeMap	O(log n)	O(log n)	O(n) traversal	O(n)
Sorted array	O(log n)	O(n) shift	O(n) traversal	O(n)

A sorted array suits a mostly static inventory with frequent lookups and code-order reports. It is less suitable when insertions/removals are frequent because elements must be shifted. Duplicate medicine-code handling and insertion-order reporting also need explicit logic.

7. Conclusion

No map is best for every requirement. HashMap is a general-purpose choice for average constant-time lookup but offers no ordering guarantee. TreeMap is appropriate when sorted code order is a primary invariant. LinkedHashMap offers a useful balance for lookups and insertion-order reporting, with sorting required only when producing the code-order report. Choose based on actual operation frequencies and repeated evidence.

8. Execution Procedure

In Task04, compile with javac PharmacyInventoryBenchmark.java and run java PharmacyInventoryBenchmark. The program creates task4_results.csv and prints averages. No keyboard input is required. Capture your own terminal screenshot in output_screenshots. This report excludes student-identifying details.